

تمرین عملی MLOps: از مدل تا تولید در پردازش زبان طبیعی (NLP)

استاد درس: سرکار خانم دکتر مهدیه قاسمی
دستیار آموزشی: حانیه خاکشور

مقدمه

تصور کنید امروز اولین روز کاری شما در یک تیم حرفه‌ای یادگیری ماشین است و مدیر پروژه می‌گوید: «می‌خواهیم یک مدل NLP بسازیم و اون رو به یک سرویس واقعی برسونیم!» این یعنی شما فقط مدل نمی‌سازید، بلکه کل مسیر از جمع‌آوری داده‌ها تا استقرار، پایش و به‌روزرسانی مدل را تجربه خواهید کرد.

در این تمرین، شما با تیم‌های ۴ نفره کار می‌کنید. هر تیم مسئول اجرای کامل یک مدل است، از جمله پیش‌پردازش داده‌ها، آموزش، مستندسازی و سرویس‌دهی. هدف اصلی این تمرین، **تجربه واقعی چرخه MLOps** و آماده کردن شما برای چالش‌های واقعی صنعت است.

در این تمرین، شما و هم‌تیمی‌هایتان وارد دنیای جذاب MLOps خواهید شد. هدف این است که نه تنها یک مدل بسازید، بلکه کل جریان داده و مدل را از مرحله جمع‌آوری داده تا استقرار و پایش مدل در محیط واقعی تجربه کنید. همان‌طور که در صنعت یادگیری ماشین می‌بینید، ساخت مدل تنها بخشی از کار است؛ چالش اصلی، نگهداری، بهبود و استقرار مدل به‌صورت قابل اطمینان و خودکار است. در طول این تمرین، شما با ابزارهایی مانند **FastAPI** و **Docker**، **Git**، **MLflow** آشنا خواهید شد و یاد می‌گیرید چطور تیمی کار کنید و پروژه‌های NLP را مدیریت کنید.

مدل پایه می‌تواند تحلیل احساسات، طبقه‌بندی متن یا اصلاح املائی باشد. اگر نمی‌خواهید از تمرین‌های گذشته استفاده کنید، یک **LSTM** یا **Transformer** کوچک روی دیتاست‌های استاندارد مثل **IMDB**، **AG News** در نظر بگیرید یا مجموعه متن فارسی تمرین سوم و یا تمرین‌های قبلی نیز قابل قبول است. تمرکز اصلی تمرین، **پیاده‌سازی زیرساخت و MLOps** است، نه ساخت مدل پیچیده.

هر هفته تمرکز بر روی یک بخش از جریان کار دارد، به همراه منابع مطالعه، فعالیت‌های عملی تیم، و بحث‌های گروهی. این تمرین ترکیبی از رویکردهای **Data-Centric** و **Model-Centric** است و شما را با مفاهیم مدرن **Large Language Models** و ابزارهای مدیریت و پایش مدل‌های زبانی آشنا میکند.

در هر فاز تمام اعضای تیم بخش مربوط به منابع رو بررسی می‌کنند. شما ابتدا فصل مربوطه و داکيومنت‌های مربوط به هر ابزار را می‌خوانید، سپس در گروه خود درباره آن بحث می‌کنید، و نهایتاً مفاهیم را در عمل پیاده‌سازی می‌کنید. در هر مرحله، یکی از اعضای گروه مسئول تدریس آن مبحث به سایر اعضا و ارائه به کل کلاس خواهد بود.

هفته اول: Reproducible ML Foundations & Data Handling

در این هفته تیم شما مثل آشپزهای یک رستوران است که همه مواد اولیه را آماده می‌کنند و مطمئن می‌شوند هیچ چیزی ناقص نباشد و کیفیت مواد اولیه تضمین شود؛ چون کیفیت نهایی مدل شما به همین مرحله وابسته است.

توضیح

هفته اول درباره‌ی آماده‌سازی محیط قابل بازتولید و مدیریت داده‌ها است. شما یاد می‌گیرید چگونه محیط توسعه خود را با Docker آماده کنید تا هر بار اجرای pipeline دقیقاً یکسان باشد و هیچ اختلافی در نسخه کتابخانه‌ها یا Python ایجاد نشود. Git هم برای version control ضروری است تا تغییرات کد و pipeline ثبت شود و در صورت نیاز قابل بازگشت باشد.

تیم‌ها داده‌ها را از منابع مختلف جمع‌آوری می‌کنند: فایل‌های CSV، API و JSON ها و پایگاه داده‌های رابطه‌ای مثل PostgreSQL. داده‌های حساس کاربران (PII) با روش‌های ناشناس‌سازی یا pseudonymization محافظت می‌شوند. بعد از آن، داده‌ها با DVC نسخه‌بندی می‌شوند تا هر تغییر ثبت شده و pipeline reproducible باقی بماند.

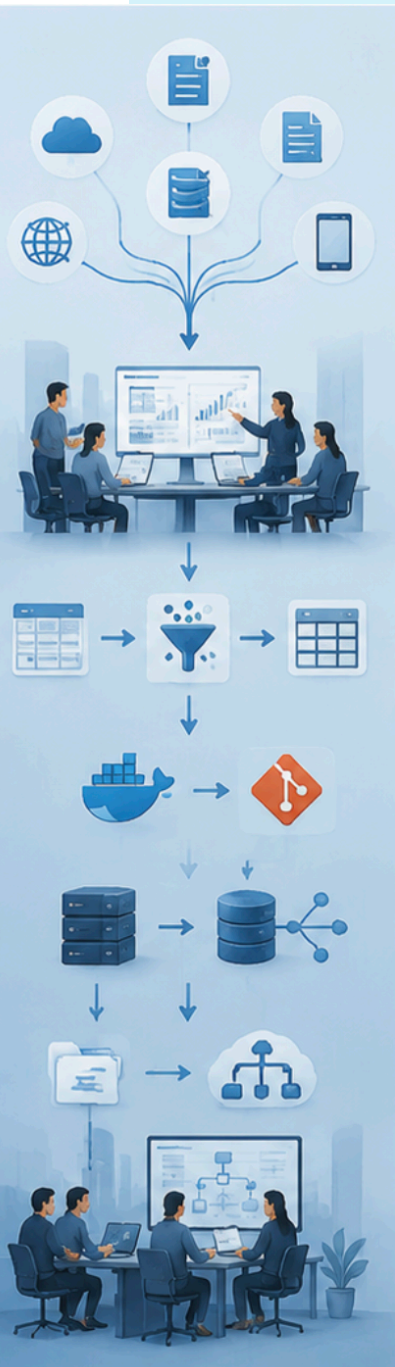
در نهایت، تیم‌ها یک pipeline ساده با Airflow یا Prefect طراحی می‌کنند تا جریان داده‌ها و وابستگی مراحل را ببینند و پایه‌ای برای هفته‌های بعد بسازند.

فعالیت تیم‌ها

- آماده‌سازی محیط Docker و ثبت نسخه‌ها است.
- Git و DVC برای کنترل نسخه داده و کد است.
- گزارش مکتوب از جلسات روزانه و مطالعه فصول مرتبط با کتاب و داکيومنت های ابزارهای مختلف.

منابع و مطالعه

- مروری بر فصول کتاب Speech and Language Processing
- ویدئوهای Docker و Git basics
- مستندات DVC و مثال DAG در Airflow/Prefect



هفته دوم: Pipelines, Reproducibility & Tracking

این هفته تیم شما مثل یک کارگاه تولید صنعتی است که خط تولید را طوری طراحی می‌کند که اگر هر قطعه خراب شود، بتوان دوباره بدون خراب شدن کل سیستم کار را ادامه داد.

توضیح

در هفته دوم، تمرکز روی ساخت pipeline های reproducible و قابل تکرار است. تیم‌ها یاد می‌گیرند داده‌ها را از preprocessing تا inference به طور مستقل مدیریت کنند و هیچ مرحله‌ای کار مرحله بعد را خراب نکند.

در این مرحله، ابتدا فصل بیستم کتاب را می‌خوانید. این فصل دو رویکرد متفاوت برای به‌روزرسانی مدل در تولید را مقایسه می‌کند. رویکرد اول، آموزش بدون حافظه (stateless) نام دارد که در آن مدل هر بار از صفر روی پنجره‌ای از داده‌های قدیم و جدید آموزش می‌بیند. رویکرد دوم، آموزش با حافظه (stateful) نام دارد که در آن مدل اولیه فقط یک بار از صفر آموزش می‌بیند و سپس با داده‌های جدید fine-tune می‌شود. شما باید تصمیم بگیرید کدام رویکرد برای مدل شما و داده‌هایتان مناسب‌تر است. عواملی که باید در نظر بگیرید شامل حجم داده‌های جدید، سرعت تغییرات، محدودیت منابع محاسباتی، و نیاز به حفظ دانش قبلی است. یک نکته کلیدی که کتاب تأکید می‌کند این است که رویکرد stateful فقط با مدل‌های قابل fine-tune مثل شبکه‌های عصبی کار می‌کند؛ مدل‌هایی مانند درخت تصمیم یا جنگل تصادفی ذاتاً stateless هستند. پس از تصمیم‌گیری، یک اسکریپت ساده می‌نویسید که فرآیند به‌روزرسانی مدل را نشان می‌دهد. اگر رویکرد stateful را انتخاب کردید، نشان می‌دهید چگونه مدل قبلی را با داده جدید fine-tune می‌کنید. اگر stateless را انتخاب کردید، نشان می‌دهید چگونه مدل را از صفر روی ترکیبی از داده‌های قدیم و جدید آموزش می‌دهید.

MLflow معرفی می‌شود تا هر پارامتر، metric و artifact مدل ثبت شود و نسخه‌های مختلف قابل مقایسه باشند. تیم‌ها همچنین با مفهوم model registry آشنا می‌شوند تا مدل‌ها از candidate به staging و نهایتاً production منتقل شوند. در کنار آن، تیم‌ها با Data-Centric AI آشنا می‌شوند. این یعنی گاهی به جای اضافه کردن پیچیدگی به مدل، کیفیت داده‌ها را افزایش می‌دهیم؛ نمونه‌های پرت حذف، برچسب‌های اشتباه اصلاح و نمونه‌های مصنوعی تولید می‌شوند. شما یاد می‌گیرید که داده‌های آموزشی خود را بررسی کنید، نمونه‌های پرت (outlier) را شناسایی و حذف کنید، برچسب‌های اشتباه را تصحیح کنید، و در صورت نیاز با تکنیک‌های داده‌سازی، نمونه‌های مصنوعی تولید کنید. کتاب اشاره می‌کند که اضافه کردن صدها مثال با کیفیت می‌تواند بهبودی برابر با میلیاردها پارامتر اضافی ایجاد کند.

فعالیت تیم‌ها

- طراحی و اجرای pipeline ETL با رعایت idempotency
- ثبت experiment ها و معیارهای مدل در MLflow
- آماده‌سازی داده‌های اصلاح‌شده و تولید نمونه‌های جدید
- گزارش مکتوب از جلسات روزانه و مطالعه فصول مرتبط با کتاب و داکيومنت های ابزارهای مختلف.

منابع و مطالعه

- فصل ۲۰ و ۲۱ کتاب Data-Centric AI و Stateless & Stateful Machine Learning Q and AI Training
- مستندات MLflow و مثال‌های model registry
- منابع pandas برای ETL reproducible



هفته سوم: سریع کردن پیش‌بینی مدل و Model Serving

حالا تیم شما مانند مدیر یک پست سریع است که باید سفارش‌ها را سریع و بدون اشتباه تحویل دهد؛ اگر مدل کند باشد، مشتری‌ها صبر نمی‌کنند.

توضیح

هفته سوم مدل شما را وارد دنیای واقعی می‌کند. مدل از MLflow استخراج شده و داخل یک REST API با FastAPI قرار می‌گیرد. اعتبارسنجی داده‌ها و مستندات خودکار API نیز اضافه می‌شوند. سپس سرویس در یک Docker container قرار می‌گیرد تا در هر محیطی بدون مشکل اجرا شود. همچنین، سرعت inference با دو تکنیک اصلی بهبود می‌یابد: batching برای اجرای همزمان چند درخواست و quantization برای کاهش حجم مدل و افزایش سرعت.

سؤالی که فصل ۲۲ پاسخ می‌دهد: اگر مدل ما برای پاسخ‌گویی به درخواست‌های کاربران کند است و هزینه سرور بالا رفته، چه تکنیک‌هایی می‌توانیم بدون تغییر معماری اصلی مدل، سرعت استنتاج را افزایش دهیم؟ چگونه می‌توان یک مدل ۱۰ میلیارد پارامتری را روی گوشی موبایل اجرا کرد؟ وقتی مدلی در تولید اجرا می‌شود، سرعت پیش‌بینی حیاتی است. فصل بیست و دوم کتاب تکنیک‌های مختلفی را معرفی می‌کند که می‌توانید بدون تغییر معماری مدل، سرعت استنتاج را افزایش دهید. این تکنیک‌ها در سه سطح دسته‌بندی می‌شوند. سطح اول، بهینه‌سازی‌های نرم‌افزاری هستند که شامل پردازش دسته‌ای (batching) و برداری‌سازی (vectorization) می‌شود. پردازش دسته‌ای یعنی چندین نمونه را همزمان به مدل بدهید تا از قابلیت موازی‌سازی GPU استفاده کنید. برداری‌سازی یعنی عملیات را روی کل آرایه انجام دهید نه با حلقه‌های جداگانه. کتاب تأکید می‌کند که این تکنیک‌ها هیچ افت دقتی ندارند. سطح دوم، بهینه‌سازی حافظه و محاسبات مانند فشرده‌سازی مدل با روش quantization است که وزن‌های مدل را از اعداد ممیز ۳۲ بیتی به اعداد صحیح ۸ بیتی تبدیل می‌کند. این کار اندازه مدل را حدود چهار برابر کاهش می‌دهد و سرعت را به شدت افزایش می‌دهد. کتاب هشدار می‌دهد که quantization گاهی باعث افت جزئی دقت می‌شود و باید آن را اندازه گرفت. سطح سوم، تکنیک‌های پیشرفته‌تری مانند pruning (حذف وزن‌های کم‌اهمیت) و knowledge distillation (آموزش یک مدل کوچک برای تقلید از مدل بزرگ) است که در صورت نیاز می‌توانید مطالعه کنید. در این مرحله، شما حداقل دو تکنیک از این تکنیک‌ها را روی مدل خود پیاده‌سازی می‌کنید. اولین قدم ثبت زمان و حافظه مصرفی مدل اصلی (baseline) است. سپس پردازش دسته‌ای را فعال می‌کنید و بهبود سرعت را اندازه می‌گیرید. در قدم بعدی، مدل را با quantization فشرده می‌کنید و اندازه مدل و سرعت پیش‌بینی را دوباره اندازه می‌گیرید. اگر مدل شما با کتابخانه PyTorch یا TensorFlow ساخته شده، quantization فراخوانی ساده تابع است. شما نتایج را در جدولی مقایسه می‌کنید و در گزارش خود توضیح می‌دهید که هر تکنیک چه میزان بهبود در سرعت و کاهش در اندازه مدل ایجاد کرده است و آیا افت دقتی مشاهده شده است یا خیر.

فعالیت تیم‌ها

- API و مستندات FastAPI
- Dockerizing سرویس و تست‌های محلی
- اندازه‌گیری و بهینه کردن سرعت و مصرف حافظه مدل
- گزارش مکتوب از جلسات روزانه و مطالعه فصول مرتبط با کتاب و داکيومنت های ابزارهای مختلف.

منابع و مطالعه

- فصل ۲۲ کتاب Machine Learning Q and AI
- مستندات FastAPI و Docker
- مقالات و ویدئوهای quantization و batching, vectorization



در این مرحله تیم شما مانند نگهبان یک باغ بزرگ است؛ باید همواره مراقب باشید که تغییرات محیط باعث آسیب به محصولات (مدل) نشود و در صورت نیاز سیستم را به‌روزرسانی کنید.

توضیح

در هفته چهارم، تیم‌ها نقش نگهبانان کیفیت مدل را برعهده می‌گیرند. CI/CD معرفی می‌شود و pipeline برای تست و build خودکار آماده می‌شوند. مدل پایش می‌شود: latency، خطاها، مصرف منابع و دقت بررسی می‌شوند.

انواع drift (covariate, label, concept) بررسی می‌شوند و تیم‌ها یک سیستم تشخیص drift پیاده‌سازی می‌کنند. همچنین یک endpoint برای جمع‌آوری feedback کاربران اضافه می‌شود تا مدل همیشه عملکرد خوبی داشته باشد و در صورت نیاز فرآیند retraining خودکار اجرا شود.

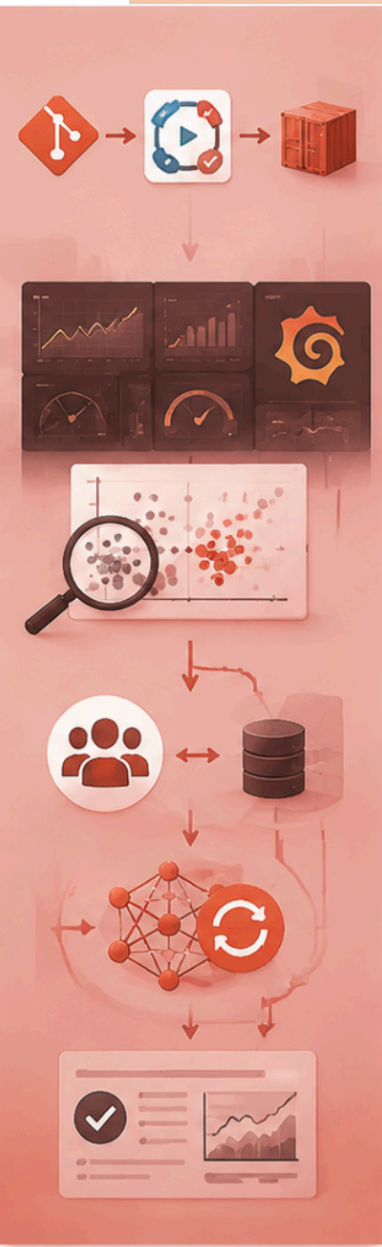
سؤالی که این فصل پاسخ می‌دهد: بعد از مدتی که مدل در تولید است، چرا کیفیت پیش‌بینی‌هایش کاهش پیدا می‌کند؟ اصلاً چه نوع تغییراتی در داده ممکن است رخ دهد و برای هر نوع چه راهکاری وجود دارد؟ چگونه می‌توان این تغییرات را زود هنگام تشخیص داد؟ یکی از چالش‌های اصلی در تولید این است که مدل شما با داده‌ای روبرو می‌شود که ممکن است با داده آموزشی تفاوت داشته باشد. فصل بیست و سوم کتاب انواع مختلف تغییر توزیع داده را معرفی می‌کند. نوع اول، تغییر در ویژگی‌های ورودی (covariate shift) است. در این حالت، توزیع ویژگی‌های ورودی $p(x)$ عوض شده اما رابطه بین ویژگی‌ها و برچسب‌ها $p(y|x)$ ثابت مانده است. مثال ساده: ایمیل‌های دریافتی طولانی‌تر شده‌اند، اما "رابطه بین طول ایمیل و اسپم بودن" تغییر نکرده است. این نوع کمترین مشکل را دارد و با روش importance weighting قابل حل است. نوع دوم، تغییر در توزیع برچسب‌ها (label shift) است. در این حالت $p(y)$ عوض شده اما $p(x|y)$ ثابت مانده است. مثال: در آموزش ۵۰٪ اسپم بود، در واقعیت فقط ۱۰٪ اسپم می‌رسد. نوع سوم، تغییر مفهوم (concept drift) است. در این حالت $p(y|x)$ عوض شده — یعنی رابطه بین ویژگی‌ها و برچسب تغییر کرده است. مثال: استراتژی جدید اسپمرها که در داده آموزش نبوده. نوع چهارم، تغییر دامنه (domain shift) است که بدترین حالت است و در آن هم $p(x)$ و هم $p(y|x)$ عوض می‌شوند. مثال: هم محتوای ایمیل عوض شده، هم تعریف spam. در این مرحله، شما یک آشکارساز تغییر (drift detector) پیاده‌سازی می‌کنید که توزیع ویژگی‌های ورودی را در طول زمان پایش می‌کند و در صورت تغییر معنادار، هشدار می‌دهد یا فرآیند به‌روزرسانی مدل را آغاز می‌کند. می‌توانید از کتابخانه alibi-detect استفاده کنید یا یک روش ساده مانند Population Stability Index (PSI) را خودتان پیاده‌سازی کنید. علاوه بر این، شما یک endpoint ساده به API خود اضافه می‌کنید که از کاربران feedback می‌گیرد. این feedback می‌تواند برای تشخیص افت کیفیت مدل در طول زمان کمک کند.

فعالیت تیم‌ها

- طراحی و مدیریت CI/CD و تست pipeline
- پیاده‌سازی drift detector و feedback collection
- پایش عملکرد مدل و گزارش مشکلات احتمالی
- گزارش مکتوب از جلسات روزانه و مطالعه فصول مرتبط با کتاب و داکيومنت های ابزارهای مختلف.

منابع و مطالعه

- فصل ۲۳ کتاب Machine Learning Q and AI
- مستندات GitHub Actions/GitLab CI
- منابع Grafana و drift detection



ارزیابی تمرین بر اساس سه معیار اصلی انجام می‌شود:

اول، تحویل به موقع و کامل تمام اجزای پروژه شامل کدها، مستندات، و مخزن **GitHub**. حتی اگر پیاده‌سازی کامل نباشد، تحویل و گزارش صادقانه از وضعیت کار ارزش بالایی دارد.

دوم، کیفیت پیاده‌سازی که با مشاهده دمو و بررسی کد ارزیابی می‌شود. آیا **API** تمیز و قابل فهم طراحی شده؟ آیا هر یک از فازها به درستی اجرا شده است؟ آیا گروه توانسته چالش‌های پیاده‌سازی را حل کند؟

سوم، کیفیت مستندات و ارائه شفاهی که شامل گزارش فازها، ارائه مفاهیم کتاب به کلاس، و دموی نهایی می‌شود.

در طول این تمرین، جلسات دوشنبه‌های کلاس مربوط به این تمرین میشود و هر یک از گروه‌ها مسئول توضیح و تدریس و تفصیل یکی از فصل‌های گفته شده هستند. در این جلسات گروه‌ها می‌توانند سؤالات خود را بپرسند و پیشرفت خود را با استاد بررسی کنند.

